

Clase #05 de 27

Tipos Flotantes & Funciones

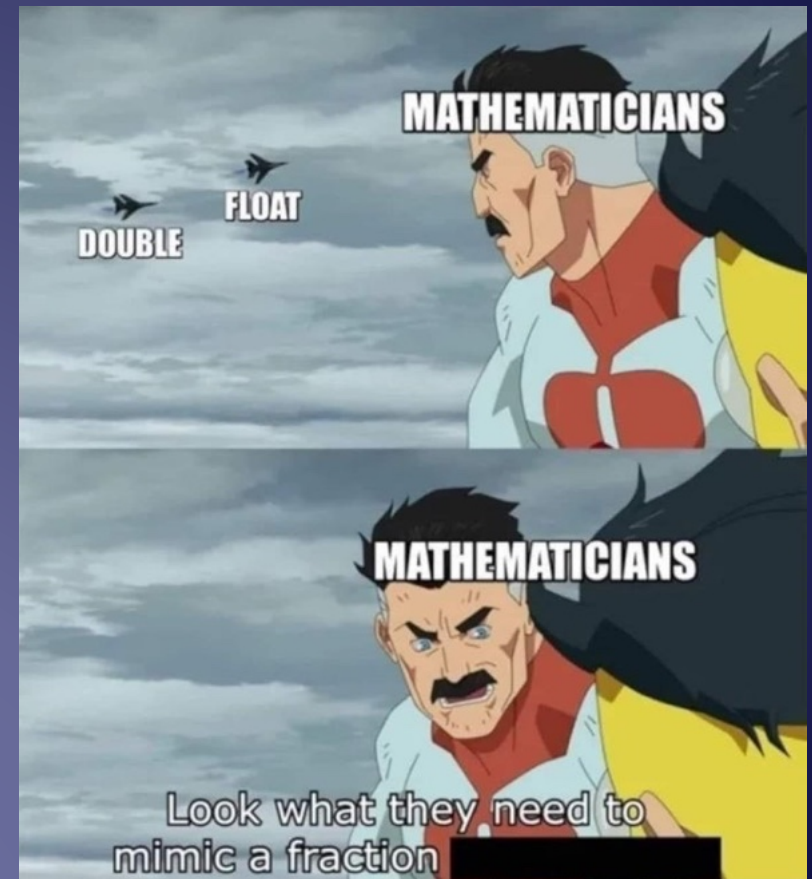
May 6, Miércoles

Agenda para esta clase

- Tipos Punto-Flotante
- Introducción a Funciones
- Proceso de Desarrollo de Funciones
- Ejemplo: Aplicación del Proceso de Desarrollo para Funciones Escalares Simples
- Ejercicio: Desarrollar Función Celsius(fahr)
- Comparación de Flotantes

Tipos Punto-Flotante

Una aproximación a los números reales

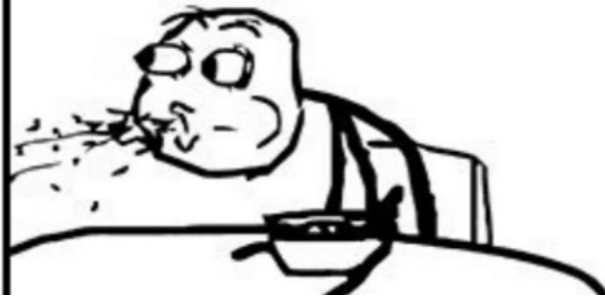


$0.1 + 0.2$



It will be 0.3

0.30000000000000004



Me after learning about
floating point inaccuracy



Representación de Reales en tipo double

- Reales es un conjunto infinito
- Se representan electrónicamente con un conjunto finito de bits, generalmente 32 o 64 bits
- Entonces, la cantidad de valores que se pueden representar es finita
- La representación se define en IEEE 60559 y IEEE 754
- Sigue la notación científica, pero en base 2
- No todos los valores reales se pueden representar precisamente en base decimal, por ejemplo $1/3$ se se aproxima con 1.333
- De la misma forma no todos los valores reales se pueden representar de forma exacta en base binaria, por ejemplo $1/10$
- De la escala continua de reales, la representación IEEE puede representar solo algunos y esa distribución no es uniforme
- En C++ hay varios tipos que representan reales, nosotros usamos double, que es el tipo por defecto
- Otros lenguajes tienen tipos equivalentes
- Es un tema que se desarrolla en la asignatura “Arquitectura de Computadoras”
- Artículos sobre el tema: [Universidad de Texas](#), [Bit Bashing](#)
- [Convertidor de Punto Floante IEEE-754](#).

Recuerden repasar para qué se usan

Uso de los Tipos Numéricos:

Tipo Natural (unsigned) versus Tipo Flotante (double)

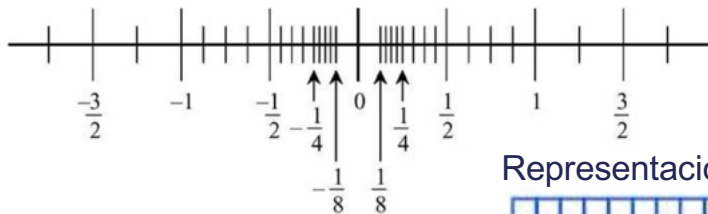
Contable

- En Matemática:
 - $\mathbb{N}$ (Natural, infinitos)
- En C++:
 - unsigned (subrango **finito**)
 - Hay otros
- Cantidades
- Discreto
- Ejemplos
 - Stock
 - Participantes de una clase
 - Unidades monetarias (centavos)

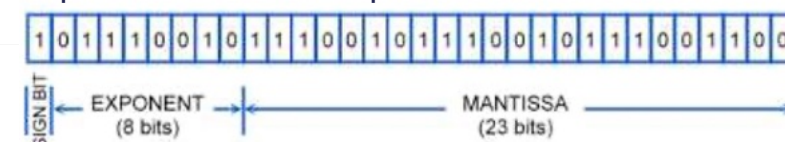
No contable

- En Matemática
 - $\mathbb{R}$ (Real, infinitos)
- En C++:
 - Aproximación con double (subconjunto **finito** de $\mathbb{R}$)
 - Hay otros
- Continuo
- Mediciones $\Rightarrow$ error
- Ejemplos
 - Temperatura de una habitación
 - Volumen de agua de un recipiente
 - Probabilidad de ocurrencia

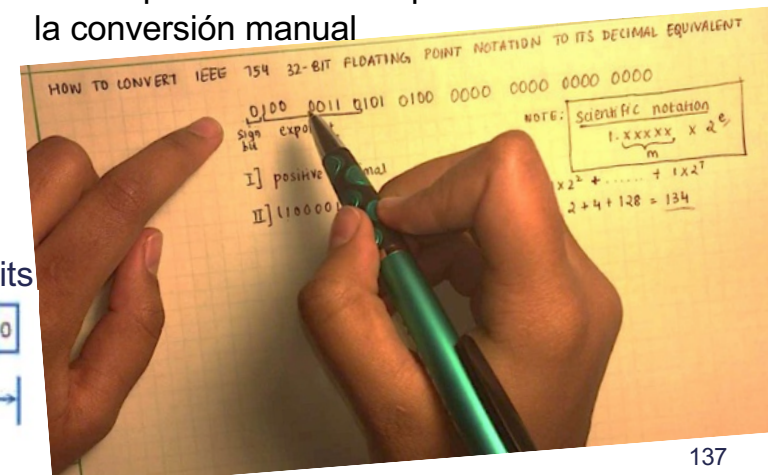
Distribución no uniforme



Representación en bloque con cantidad finita de bits



En “Arquitectura de Computadoras” realizan la conversión manual



Ejemplo de Representación de algunos Valores

[illegible]

$$1 \times 2^{-1022} \times 0 = 0$$

= 0x3FF0000000000000

$$1 \times 2^0 \times 1 = 1$$

= 0xBFF0000000000000

$$-1 \times 2^0 \times 1 = -1$$

= 0x4000000000000000

$$1 \times 2^1 \times 1 = 2$$

= 0xC000000000000000

$$-1 \times 2^1 \times 1 = -2$$

= 0x4014000000000000

$$1 \times 2^2 \times 1.25 = 5$$

= 0x3FE0000000000000

$$1 \times 2^{-1} \times 1 = 0.5$$

= 0x3FB9999999999999A

$$1 \times 2^{-4} \times 1.6 = 0.1$$

[illegible]

$$1 \times 2^{-3} \times 1.6 = 0.2$$

[illegible]

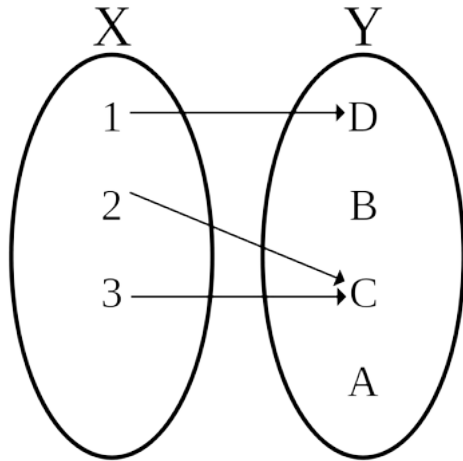
$$1 \times 2^{-2} \times 1.2 = 0.3$$

Tipos Flotantes en C++

- Existen varios tipos flotantes en C++
 - Tipos Punto Flotante Estándar
 - float 3.14F 3.14f
 - double 3.14
 - long double 3.14L 3.14l
 - Tipos Punto Flotante de Ancho fijo, opcionales
 - float16_t 3.14F16 3.14f16
 - float32_t 3.14F32 3.14f32
 - float64_t 3.14F64 3.14f64
 - float128_t 3.14F128 3.14f128
 - bfloat16_t 3.14BF16 3.14bf16
- Solo vamos a usar double por se el tipo flotante por defecto

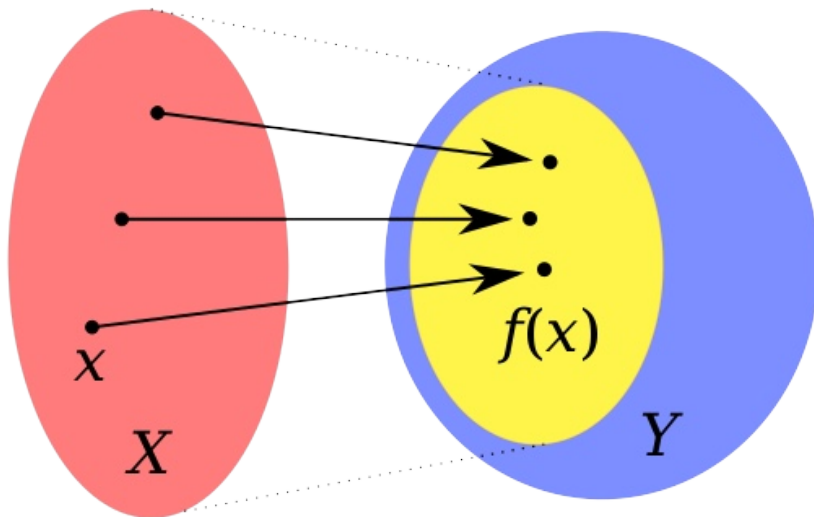
Introducción a Funciones

Funciones en Matemática y en Lenguajes de Programación: Ejemplo Introductorio



$$f: \mathbb{R} \rightarrow \mathbb{R} / f(x) = 2x + 1$$

```
double f(double);  
double f(double x){ return 2*x + 1; }
```



$$f: X \rightarrow Y$$

- Conjunto de Salida y Dominio
- Conjunto de Llegada y Codominio

Programa Completo

```
#include<cassert>
#include<iostream>
```

```
double f(double);
```

```
int main(){
    std::cout << f(3);
    assert( 2*3+1 == 7 );
    assert( 7 == f(3) );
}
```

```
double f(double x){return 2*x+1;}
```

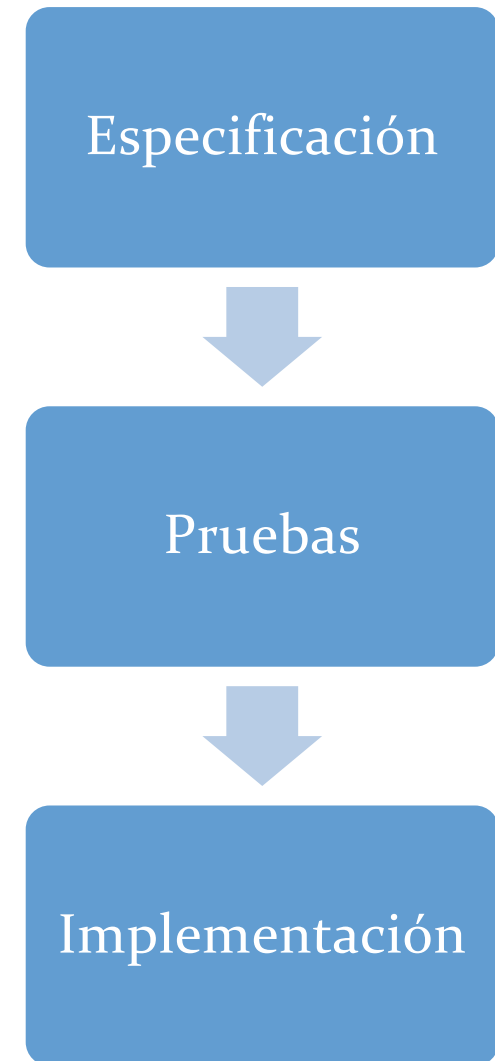
$$f: \mathbb{R} \rightarrow \mathbb{R} / f(x) = 2x + 1$$

- `assert` para pruebas, no vamos a usar `cin` o `cout` para probar
- Casos de prueba, partición de conjunto de valores
- Terminología
 - Declaración o prototipo de la función
 - Invocación
 - Argumentos
 - Definición o Implementación de la función
 - Parámetros
- Analogía con Matemática

Proceso de Desarrollo de Funciones

Proceso de Desarrollo de Funciones

- Proceso General
 1. Definir una *especificación* matemática de una función.
 2. Definir pruebas en base a la especificación.
 3. Programar una *implementación* en base a la especificación.
- Proceso Detallado
 1. Especificación de la función.
 2. Diseño de Pruebas, con partición del conjunto de los datos de prueba
 3. Codificación de las pruebas.
 4. Compilación de las pruebas.
 5. Implementación de la función.
 6. Compilación de la implementación.
 7. Ejecución.
 8. Evaluación de resultados.



Ejemplo: Aplicación del Proceso de Desarrollo para Funciones Escalares Simples

Constante

```
#include <cassert>
#include <iostream>

int y(int);

int main(){
    std::cout << y(-9);
    assert( 42 == y(1) );
}

int y(int x){return 42;}
```

$$y: \mathbb{Z} \rightarrow \mathbb{Z} / y(x) = 42$$

Identidad

$$id: \mathbb{Z} \rightarrow \mathbb{Z} / id(x) = x$$

```
#include <cassert>
#include <iostream>

int Id(int);

int main(){
    std::cout << Id(42);
    assert( 7 == Id(7) );
}

int Id(int x){return x;}
```

Sucesor

```
int Suc(int);
```

$$\text{suc} : \mathbb{Z} \rightarrow \mathbb{Z} / \text{suc}(x) = x + 1$$

```
assert( -41 == Suc(-42) );
```

```
assert(  0 == Suc( -1) );
```

```
assert(  1 == Suc(  0) );
```

```
assert( 42 == Suc( 41) );
```

```
int Suc(int x){return x+1;}
```

Negación (ó Inverso u Opuesto)

$$\text{neg}:\mathbb{Z} \rightarrow \mathbb{Z} / \text{neg}(x) = -x = (-1) \cdot x = 0 - x$$

```
int Neg(int);
```

```
assert( -7 == Neg( 7) );
```

```
assert(  0 == Neg(  0) );
```

```
assert( 42 == Neg(-42) );
```

```
int Neg(int x){return -x;} // return -1*x;  
                          // return 0-x;
```


Ejercicio: Desarrollar Función *Celsius(fahr)*

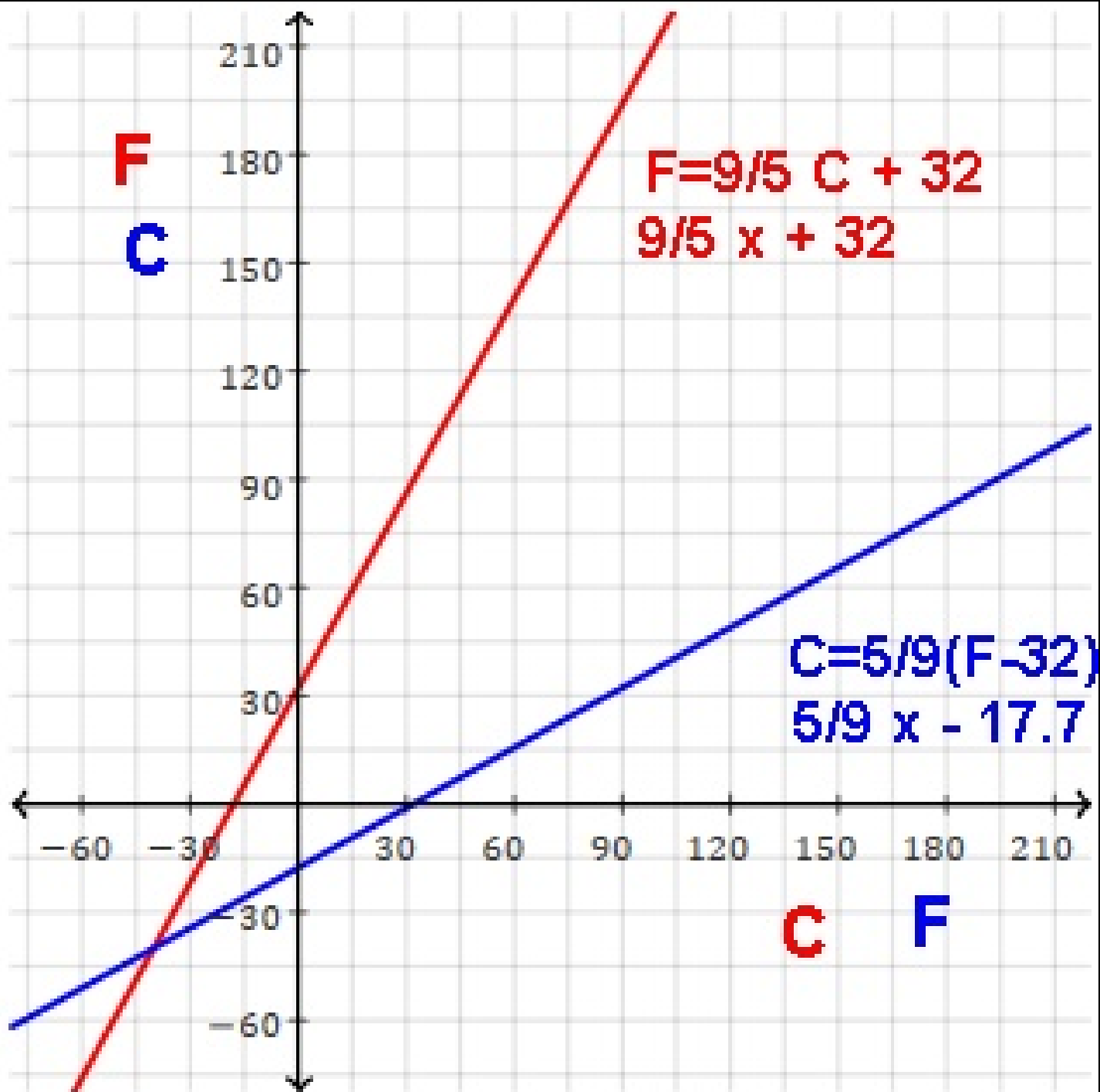
Operaciones Cerradas y Promoción de Tipo

Problema: Conversión de Fahrenheit a Celsius

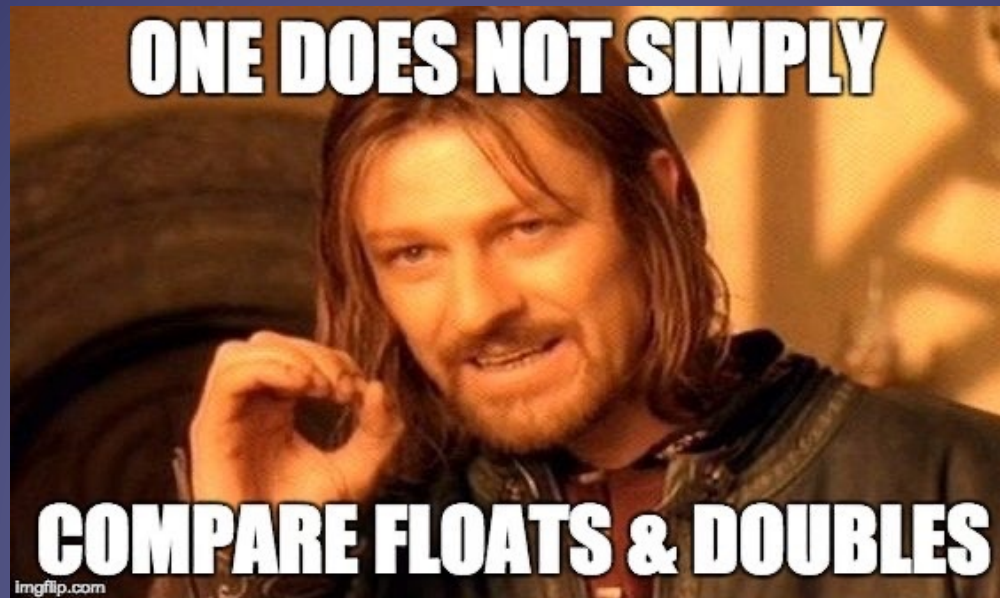
$$\text{celsius} : \mathbb{R} \rightarrow \mathbb{R} / \text{celsius}(f) = \frac{5}{9}(f - 32)$$

```
double celsius(double);
```

- Pero hay dos sub-problemas a solucionar antes de poder probar e implementar la función:
 - Valor del racioal cinco novenos versus la división entera en C++ de la expresión 5/9
 - Representación inexacta de los tipos flotantes
- Una solución al primer problema es realizar división entre flotantes.
- Para el segundo problema, debemos incorporar la comparación con tolerancia.



Comparación de Flotantes



Comparación de Flotantes

Problema

- Los valores reales se guardan en los registros de la CPU como **aproximaciones representadas en base binaria**, por eso, valores como 1.0, 2.0, y 1.5, 0.25 que se presentan de forma exacta, mientras otros no, por ejemplo 0.1, 0.2, 0.3.
- Por eso, las siguientes aseveraciones son correctas:

```
assert( 0.1 + 0.2      != 0.3 );  
assert( 1.0 / 3.0      != 0.333333333 );  
assert( 0.333333333 * 3.0 != 1.0 );
```

Solución

- Comparar dentro de un entorno, o rango, con un tolerancia o margen de error.
- Es decir, buscamos determinar si dos números son casi iguales o suficientemente iguales, si están cerca entre ellos o en un determinado entorno o rango.
- Esa tolerancia la podemos llamar diferencia, *delta*, error, o *épsilon*
- La comparación se va a abstraer en una **función**.

Nombre de la Función de Comparación de Flotantes: *Bastante Cerca*

- La función abstrae una comparación que considera la aproximación, algunos nombres válidos son los siguientes, la elección puede ser subjetiva o basarse en el estilo de programación del resto del proyecto

Igual	- Equal - EpsilonEqual - FloatEqual - NearlyEqual - AlmostEqual - ApproximatelyEqual - EqualEnough	- Igual - IgualÉpsilon - IgualFlotante - Casilgual - AproximadamenteIgual - Suficientementelgual
Predicados	- AreAlmostEqual - IsAlmostEqual - AreNear - IsNear - AreNearlyEqual - AreApproximatelyEqual - ArePracticallyTheSame	- SonCasilguales - EsCasilgual - EstánCerca - EstáCerca - SonCasilguales - SonAproximadamentelguales - SonPráctciamnteLoMismo
Otros	- Aprox - Close - CloseEnough - WithinTolerance	- Aprox - Cerca - BastanteCerca - DentroDeLaTolerancia

- Los predicados se pueden prefijar, entre otros, con estas palabras: *is*, *are*, *es*, *son*.
- Es común aceptar la partícula *is* como indidacador de predicado más allá del *idioma* y la *gramática*.

Especificación: *Cerca*

- En este ejemplo usamos un nombre breve pero claro: *Cerca*
- Tenemos una versión con ε como constante y otra como parámetro

$$\text{Cerca}(3.1416, 3.14159) = V$$

$$\text{Cerca}(3.1416, 3.14159, 0.001) = V$$

$$\text{Cerca}:\mathbb{R} \times \mathbb{R} \rightarrow \text{B/}$$

$$\text{Cerca}:\mathbb{R} \times \mathbb{R} \times \mathbb{R} \rightarrow \text{B/}$$

$$\text{Cerca}(a, b) = (a - \varepsilon) \leq b \leq (a + \varepsilon) \quad \text{Cerca}(a, b, \varepsilon) = (a - \varepsilon) \leq b \leq (a + \varepsilon)$$

$$\text{Cerca}(a, b) = |a - b| \leq \varepsilon$$

$$\text{Cerca}(a, b, \varepsilon) = |a - b| \leq \varepsilon$$

Implementación: *Cerca*

```
bool Cerca(double, double, double ); // Los nombres de los parámetros
bool Cerca(double a, double b, double ε ); // son opcionales, inclusive
bool Cerca(double a, double b, double ε = 0.001); // para el valor por defecto
```

```
assert( not Cerca( 1.0, 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1, 0.0) ); // 1 == 10*(1/10)
assert( Cerca( 1.0, 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 ) );
assert( Cerca( 42.0, 42.0 | , 0.0 ) );
assert( Cerca( 1.0, 0.999 ) );
assert( Cerca( 1.0, 0.9 , 0.1 ) );
assert( not Cerca( 0.1 + 0.2, 0.3 , 0.0 ) );
assert( Cerca( 0.1 + 0.2, 0.3 ) );
assert( Cerca( 1.0/3.0, 0.333 ) );
assert( not Cerca( 1.0/3.0, 0.33 ) );
```

```
bool Cerca(double a, double b, double ε){ // es el epsilon, delta, error, o tolerancia para la comparación
    return (a-ε) <= b and b <= (a+ε);
}
```

Uso de Cerca para probar Celsius

```
#include <cassert>
#include <iostream>
```

```
double Celsius(double);
bool Cerca(double, double, double = 0.001);
```

```
int main(){
    // Celsius test 1
    assert( 0 == Celsius(32) );
    std::cout << Celsius(64); // 17.7778
    assert( 17.7778 != Celsius(64) );
    assert( 17.777 < Celsius(64) );
    assert( 17.778 > Celsius(64) );
    assert( 17.777 < Celsius(64) and Celsius(64) < 17.778);

    // Celsius test 2
    assert( Cerca(-35.5556, Celsius(-32)) );
    assert( Cerca(-17.7778, Celsius( 0)) );
    assert( Cerca( 17.7778, Celsius( 64)) );
    assert( Cerca( 17.77777777778, Celsius( 64), 0.000000000001 ) );
}
```

```
double Celsius(double f){ return 5.0/9.0*(f-32);}
```

```
bool Cerca(double a, double b, double ε){ return (a-ε) <= b and b <= (a+ε);}
```

Why $0.1 + 0.2 \neq 0.3$ in JavaScript

`Console.log(0.1 + 0.2)` Floating-point arithmetic

IEEE 754 standard Binary fractions Rounding errors

Epsilon comparison `toFixed()` `Math.round()`

Decimal.js library Big.js Arbitrary-precision arithmetic

Binary64 format Subnormal numbers
Denormalized precision

Preguntas de Repaso

1. ¿Por qué hay infinitos valores reales pero una cantidad finita de valores de punto flotante?
2. ¿Por qué se llama flotante?
3. ¿Cuál es el tipo flotante que usamos?
4. ¿Como se declara en C++ el concepto matemático de relación $f:A \rightarrow B$?
5. ¿Como se define en C++ la fórmula de una función?
6. ¿Cuales son los tres pasos básicos del proceso de desarrollo de funciones?
7. ¿Qué valor da la expresión $5/9$? ¿Por qué?
8. ¿Cuales es un ejemplo de promoción de tipo de datos?
9. ¿Cómo comparamos valores de punto flotante? ¿Por qué?

Términos de la clase

Definir cada término con la bibliografía

- Puntos Flotantes
- Introducción a Funciones
 - Función en Matemática y en Programación
 - Definición del concepto matemático
 - Notación matemática
 - Conjunto de Salida y Dominio
 - Conjunto de Llegada y Codominio
 - Imagen
 - Prototipo o Declaración de la función
 - Definición o Implementación de la función
 - Casos de pruebas
 - Invocación
 - Argumentos
 - Parámetros
- Proceso de desarrollo de Funciones
 - Especificación
 - Prueba
 - Implementación
- Función Constante
- Función Identidad
- Función Sucesor
- Función Negación, o Inversa u Opuesto
- Múltiples implementaciones para una misma especificación
- Comparación de Flotantes
 - Truncamiento por división entera
 - Representación no precisa de tipos flotantes
 - Comparación con tolerancia
 - Argumentos por defecto

Tareas para la próxima clase

1. Idear y desarrollar funciones simples
2. Desarrollar la función Celsius; no es entrega formal, todavía.

¿Consultas?

Fin de la clase